# RA1. Programació per Components amb REACT

## Material d'Estudi i Exercicis Pràctics

## BLOC 1: Introducció - Paradigma de Components i React

📚 Teoria

**Què és React?**

React és una biblioteca de JavaScript per construir interfícies d'usuari basada en components. Creada per Facebook el 2013, React utilitza un paradigma declaratiu on descrivim **què** volem veure, no **com** fer-ho.

**Conceptes Clau**

**1. Components**

- Peces reutilitzables i independents de la UI
- Encapsulen estructura, estil i comportament
- Es poden combinar per crear interfícies complexes

**2. JSX (JavaScript XML)**

- Extensió de sintaxi que permet escriure HTML dins de JavaScript/TypeScript
- Es transpila a crides a `React.createElement()`
- Permet interpolació amb `{}`

```
// JSX
const element = <h1>Hola, {nom}</h1>;

// Es transpila a:
const element = React.createElement('h1', null, 'Hola, ', nom);
```

**3. Virtual DOM**

- Representació en memòria de la UI real
- React compara versions per detectar canvis (reconciliació)
- Només actualitza el DOM real amb els canvis necessaris

**4. Flux Unidireccional de Dades**

- Les dades flueixen dels components pare als fills via props
- Mai al revés directament
- Millora la predictibilitat i facilita el debugging

**5. Composició sobre Herència**

- React afavoreix combinar components petits
- Millor que jerarquies de classes complexes

**Tipus de Components**

**Components Funcionals (Modern)**

```
const Welcome = ({ name }: { name: string }) => {
  return <h1>Hola, {name}</h1>;
};
```

**Components de Classe (Legacy)**

```
class Welcome extends React.Component {
  render() {
    return <h1>Hola, {this.props.name}</h1>;
  }
}
```

---

## ✏️ Exercicis Bloc 1

**Exercici 1.1: Primer Component (Bàsic)**

Crea un component `Greeting` que mostri un missatge de benvinguda.

**Requisits:**

- Ha de rebre un prop `name` de tipus string
- Ha de mostrar "Benvingut/da, [name]!"
- Usa TypeScript per tipar els props

```
// Ús
<Greeting name="Maria" />
```

---

**Exercici 1.2: Component amb Múltiples Props (Bàsic)**

Crea un component `UserInfo` que mostri informació d'un usuari.

**Requisits:**

- Props: `name` (string), `age` (number), `email` (string)
- Mostra la informació en un format llegible
- Tipa correctament amb una interface

```
// Exemple d'ús
<UserInfo name="Joan" age={25} email="joan@example.com" />
```

---

#### Exercici 1.3: Component amb Props Opcionals (Intermedi)

Crea un component `Avatar` per mostrar una imatge de perfil.

**Requisits:**

- Prop `src` (string, opcional)
- Prop `alt` (string)
- Prop `size` (number, opcional, default: 50)
- Si no hi ha `src`, mostra les inicials del nom en un cercle de color
- Usa props opcionals amb `?`

```
// Exemple d'ús
<Avatar src="/user.jpg" alt="Maria Garcia" size={100} />
<Avatar alt="Joan Puig" /> // Sense imatge, mostra inicials "JP"
```

---

#### Exercici 1.4: Llista de Components (Intermedi)

Crea un component `TaskList` que mostri una llista de tasques.

**Requisits:**

- Rep un array de tasques: `{ id: number, title: string, completed: boolean }`
- Renderitza cada tasca com un element de llista
- Les tasques completades han de tenir un estil diferent (tatxades)
- Utilitza `.map()` per recórrer l'array
- Afegeix la prop `key` correctament

```
interface Task {
  id: number;
  title: string;
  completed: boolean;
}

interface TaskListProps {
  tasks: Task[];
}
```

---

#### Exercici 1.5: Composició de Components (Avançat)

Crea un sistema de `Card` components reutilitzables.

**Requisits:**

- Component `Card` (contenidor principal)
- Component `CardHeader` (títol i subtítol opcionals)
- Component `CardBody` (contingut)
- Component `CardFooter` (accions opcionals)
- Usa `children` per permetre composició flexible
- Exemple d'ús:

```
<Card>
  <CardHeader title="Producte" subtitle="Descripció breu" />
  <CardBody>
    <p>Contingut del producte aquí</p>
  </CardBody>
  <CardFooter>
    <button>Comprar</button>
  </CardFooter>
</Card>
```

---

## BLOC 2: Entorn React + TypeScript

### 📚 Teoria

**Configuració amb Vite**

Vite és la eina moderna recomanada per projectes React el 2026.

**Crear projecte:**

```
npm create vite@latest nom-projecte
# Seleccionar: React → TypeScript
cd nom-projecte
npm install
npm run dev
```

**Avantatges de Vite:**

- Arrancada instantània
- Hot Module Replacement (HMR) ultràpid
- Build optimitzat amb Rollup
- Configuració mínima

**Estructura de Projecte Escalable**

```
src/
├── components/      # Components reutilitzables
│   ├── Button/
│   │   ├── Button.tsx
│   │   ├── Button.test.tsx
│   │   └── index.ts
│   └── index.ts     # Barrel export
├── pages/           # Vistes principals
├── hooks/           # Custom hooks
├── services/        # API calls
├── types/           # Interfícies TypeScript globals
├── utils/           # Funcions auxiliars
├── styles/          # Estils globals
└── App.tsx
```

**Sistema de Mòduls ES6**

**Named Exports:**

```
// utils.ts
export const formatPrice = (price: number) => `${price}€`;
export const formatDate = (date: Date) => date.toLocaleDateString();

// App.tsx
import { formatPrice, formatDate } from './utils';
```

**Default Exports:**

```
// Button.tsx
const Button = () => <button>Click</button>;
export default Button;

// App.tsx
import Button from './Button';
```

**Re-exports (Barrel Pattern):**

```
// components/index.ts
export { Button } from './Button';
export { Card } from './Card';
export { Input } from './Input';

// App.tsx
import { Button, Card, Input } from '@/components';
```

### TypeScript en React

#### Tipar Props:

```
interface ButtonProps {
  text: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
  disabled?: boolean;
}

const Button = ({ text, onClick, variant = 'primary', disabled }:
ButtonProps) => {
  return <button onClick={onClick} disabled={disabled}>{text}</button>;
};
```

#### Tipar Events:

```
const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {
  console.log(e.currentTarget);
};

const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  console.log(e.target.value);
};

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
};
```

#### Children i ReactNode:

```
import { ReactNode } from 'react';

interface ContainerProps {
  children: ReactNode;
  className?: string;
}

const Container = ({ children, className }: ContainerProps) => {
  return <div className={className}>{children}</div>;
};
```

### Configuració Avançada

#### vite.config.ts amb path aliases:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});
```

**tsconfig.json per paths:**

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

---

## ✏️ Exercicis Bloc 2

**Exercici 2.1: Configurar Projecte (Bàsic)**

Configura un projecte React + TypeScript des de zero.

**Tasques:**

1. Crear projecte amb Vite
2. Crear estructura de carpetes: components/, types/, utils/
3. Configurar path alias `@` → `./src`
4. Crear un fitxer `types/index.ts` amb una interface `User`
5. Importar-la des d'un component usant `@/types`

---

**Exercici 2.2: Sistema de Mòduls (Bàsic)**

Crea un mòdul d'utilitats amb múltiples funcions.

**Fitxer:** `utils/formatters.ts`

- `formatPrice(price: number): string` - Formata preus amb €
- `formatDate(date: Date): string` - Format DD/MM/YYYY

- `capitalize(str: string): string` - Primera lletra majúscula

Exporta-les amb named exports i utilitza-les en un component.

---

#### Exercici 2.3: Barrel Exports (Intermedi)

Organitza múltiples components amb barrel exports.

**Estructura:**

```
components/
├── Button/
│   ├── Button.tsx
│   └── index.ts
├── Card/
│   ├── Card.tsx
│   └── index.ts
├── Input/
│   ├── Input.tsx
│   └── index.ts
└── index.ts  # Barrel export principal
```

**Requisits:**

- Cada component té el seu directori
- Cada directori té un `index.ts` que fa re-export
- El `components/index.ts` re-exporta tots els components
- Importa'ls en `App.tsx` amb una sola línia

---

#### Exercici 2.4: Component ProductCard amb TypeScript (Intermedi)

Crea un component `ProductCard` totalment tipat.

**Interface:**

```
interface Product {
  id: number;
  name: string;
  price: number;
  image?: string;
  inStock: boolean;
  category: 'electronics' | 'clothing' | 'food';
}

interface ProductCardProps {
  product: Product;
  onAddToCart: (id: number) => void;
}
```

**Requisits:**

- Mostrar tota la informació del producte
- Botó "Afegir a la cistella" deshabilitat si `inStock` és false
- Usar el tipus correcte per l'event handler
- Aplicar estils diferents segons category

---

**Exercici 2.5: Formulari de Contacte Tipat (Avançat)**

Crea un formulari de contacte amb validació i TypeScript complet.

**Requisits:**

- Interface `ContactForm`: name, email, phone?, message
- useState per gestionar l'estat del formulari
- Tipus correctes per tots els events (onChange, onSubmit)
- Validació abans d'enviar (email format, camps requerits)
- Mostrar errors de validació
- Reset del formulari després d'enviar

```
interface ContactFormData {
  name: string;
  email: string;
  phone?: string;
  message: string;
}

interface FormErrors {
  name?: string;
  email?: string;
  message?: string;
}
```

---

## BLOC 3: Estat i Hooks

### 📚 Teoria

**Què és l'Estat?**

L'estat és la memòria del component. Quan canvia l'estat, React torna a renderitzar el component automàticament.

**Característiques:**

- Privat i totalment controlat pel component
- Immutable: mai modificar directament

- Asíncron: les actualitzacions poden ser agrupades

#### Hook useState

El hook més bàsic per gestionar estat en components funcionals.

**Sintaxi bàsica:**

```
const [value, setValue] = useState<tipus>(valorInicial);
```

**Exemples:**

```
// Tipus inferit
const [count, setCount] = useState(0);

// Tipus explícit
const [user, setUser] = useState<User | null>(null);

// Array d'objectes
const [items, setItems] = useState<Item[]>([]);

// Objecte complet
const [form, setForm] = useState<FormData>({
  name: '',
  email: ''
});
```

#### Actualitzar Estat

**Valor directe:**

```
setCount(5);
setUser({ id: 1, name: 'Maria' });
```

**Funció actualitzadora (recomanat per basar-se en valor anterior):**

```
setCount(prev => prev + 1);
setItems(prev => [...prev, newItem]);
```

**Actualitzar objectes (immutabilitat):**

```
// ❌ Mal - Muta l'estat directament
user.name = 'Nou nom';
```

```
setUser(user);

// ✅ Bé - Crea nou objecte
setUser({ ...user, name: 'Nou nom' });

// Actualitzar propietat nested
setForm(prev => ({
  ...prev,
  address: { ...prev.address, city: 'Barcelona' }
}));
```

**Hook useEffect**

Executa efectes secundaris (side effects) després del render.

**Sintaxi:**

```
useEffect(() => {
  // Codi a executar

  return () => {
    // Cleanup (opcional)
  };
}, [dependencies]);
```

**Casos d'ús:**

**1. Executar una vegada (componentDidMount):**

```
useEffect(() => {
  console.log('Component muntat');
}, []);
```

**2. Executar quan canvia una variable:**

```
useEffect(() => {
  console.log('Count ha canviat:', count);
}, [count]);
```

**3. Cleanup (componentWillUnmount):**

```
useEffect(() => {
  const timer = setInterval(() => {}, 1000);
```

```
    return () => clearInterval(timer); // Cleanup
  }, []);
```

**4. Fetch de dades:**

```
useEffect(() => {
  const fetchData = async () => {
    const response = await fetch('/api/users');
    const data = await response.json();
    setUsers(data);
  };

  fetchData();
}, []);
```

**Altres Hooks Importants**

**useRef** - Referència que persisteix entre renders:

```
const inputRef = useRef<HTMLInputElement>(null);

const focusInput = () => {
  inputRef.current?.focus();
};

return <input ref={inputRef} />;
```

**useMemo** - Memoritza valors calculats:

```
const expensiveValue = useMemo(() => {
  return items.filter(item => item.active).length;
}, [items]);
```

**useCallback** - Memoritza funcions:

```
const handleClick = useCallback(() => {
  console.log(count);
}, [count]);
```

**Custom Hooks**

Pots crear els teus propis hooks per reutilitzar lògica.

**Exemple: useCounter**

```
const useCounter = (initialValue = 0) => {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset };
};

// Ús
const MyComponent = () => {
  const { count, increment, decrement, reset } = useCounter(10);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
};
```

---

## ✏️ Exercicis Bloc 3

**Exercici 3.1: Comptador Bàsic (Bàsic)**

Crea un component comptador amb useState.

**Requisits:**

- Estat `count` inicialitzat a 0
- Botó "Increment" (+1)
- Botó "Decrement" (-1)
- Botó "Reset" (torna a 0)
- Mostrar el valor actual

---

**Exercici 3.2: Toggle i Condicional (Bàsic)**

Crea un component que mostri/oculti contingut.

**Requisits:**

- Estat `isVisible` (boolean)
- Botó "Mostrar/Ocultar"

- Quan `isVisible` és true, mostra un paràgraf de text
- El text del botó ha de canviar segons l'estat

---

**Exercici 3.3: Input Controlat (Intermedi)**

Crea un input controlat que mostri el valor en temps real.

**Requisits:**

- Input de text controlat per useState
- Mostrar el valor actual a sota
- Mostrar la longitud del text
- Botó "Netejar" que buida l'input
- Tipar correctament l'event handler

---

**Exercici 3.4: Llista de Tasques amb useState (Intermedi)**

Crea una TODO list completa.

**Requisits:**

- Estat: array de tasques `{ id: number, text: string, completed: boolean }`
- Input per afegir noves tasques
- Botó per marcar com a completada (toggle)
- Botó per eliminar tasca
- Mostrar nombre total de tasques i completades
- Utilitzar funcions actualitzadores

```
interface Todo {
  id: number;
  text: string;
  completed: boolean;
}
```

---

**Exercici 3.5: Formulari Multi-Camp (Intermedi)**

Crea un formulari amb múltiples camps controlats.

**Requisits:**

- Camps: name, email, password, confirmPassword
- Tots controlats per un únic objecte d'estat
- Funció `handleChange` genèrica per tots els inputs
- Validació: passwords coincidents, email vàlid
- Mostrar errors de validació
- Deshabilitar botó "Submit" si hi ha errors

---

#### Exercici 3.6: useEffect - Rellotge Digital (Intermedi)

Crea un rellotge que s'actualitzi cada segon.

**Requisits:**

- useState per l'hora actual
- useEffect amb setInterval per actualitzar cada 1000ms
- Cleanup per netejar l'interval
- Format HH:MM:SS
- Botó per aturar/reprendre el rellotge

---

#### Exercici 3.7: useEffect - Fetch de Dades (Avançat)

Crea un component que carregui usuaris des d'una API.

**Requisits:**

- Usa JSONPlaceholder API: `https://jsonplaceholder.typicode.com/users`
- Estats: `users`, `loading`, `error`
- useEffect per fer el fetch al muntar
- Mostrar "Carregant..." mentre loading
- Mostrar error si falla
- Llistar usuaris quan carreguen
- Tipar correctament la resposta

```
interface User {
  id: number;
  name: string;
  email: string;
  phone: string;
}
```

---

#### Exercici 3.8: Custom Hook useLocalStorage (Avançat)

Crea un hook personalitzat per sincronitzar estat amb localStorage.

**Requisits:**

```
const useLocalStorage = <T,>(key: string, initialValue: T) => {
  // useState que llegeix de localStorage inicialment
  // Funció setValue que actualitza estat i localStorage
  // Return [value, setValue]
};
```

```
// Ús:
const [name, setName] = useLocalStorage('userName', '');
```

- Llegir de localStorage a l'inicialitzar
- Guardar a localStorage quan canvia
- Suportar tipus genèrics
- Manejar errors de serialització

---

**Exercici 3.9: Custom Hook useDebounce (Avançat)**

Crea un hook que retardi l'actualització d'un valor.

**Requisits:**

```
const useDebounce = <T,>(value: T, delay: number): T => {
  // Retorna el valor després del delay especificat
};

// Ús: cerca amb debounce
const [searchTerm, setSearchTerm] = useState('');
const debouncedSearch = useDebounce(searchTerm, 500);

useEffect(() => {
  // Fer fetch només quan debouncedSearch canvia
}, [debouncedSearch]);
```

---

## BLOC 4: Comunicació entre Components

### 📚 Teoria

**Flux de Dades en React**

React segueix un flux unidireccional de dades: **de pares a fills**.

**Principis:**

1. Les dades baixen via props
2. Els events pugen via callbacks
3. L'estat viu al component pare comú més proper

**Comunicació Pare → Fill (Props)**

El pare passa dades als fills via props.

```
// Pare
const Parent = () => {
```

```
  const [count, setCount] = useState(0);

  return <Child value={count} />;
};

// Fill
interface ChildProps {
  value: number;
}

const Child = ({ value }: ChildProps) => {
  return <p>Valor: {value}</p>;
};
```

**Comunicació Fill → Pare (Callbacks)**

El fill comunica al pare cridant funcions passades com a props.

```
// Pare
const Parent = () => {
  const [count, setCount] = useState(0);

  const handleIncrement = () => {
    setCount(prev => prev + 1);
  };

  return (
    <div>
      <p>Count: {count}</p>
      <Child onIncrement={handleIncrement} />
    </div>
  );
};

// Fill
interface ChildProps {
  onIncrement: () => void;
}

const Child = ({ onIncrement }: ChildProps) => {
  return <button onClick={onIncrement}>Incrementar</button>;
};
```

**Lifting State Up (Elevar l'Estat)**

Quan diversos components necessiten compartir estat, l'elevem al pare comú.

**Exemple: Dos inputs sincronitzats**

```
const Parent = () => {
  const [value, setValue] = useState('');

  return (
    <div>
      <Input1 value={value} onChange={setValue} />
      <Input2 value={value} onChange={setValue} />
    </div>
  );
};

interface InputProps {
  value: string;
  onChange: (value: string) => void;
}

const Input1 = ({ value, onChange }: InputProps) => {
  return (
    <input
      value={value}
      onChange={(e) => onChange(e.target.value)}
    />
  );
};
```

**Composition (children prop)**

Passar components com a fills permet composició flexible.

```
interface CardProps {
  children: ReactNode;
  title: string;
}

const Card = ({ children, title }: CardProps) => {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">
        {children}
      </div>
    </div>
  );
};

// Ús
<Card title="Producte">
  <p>Descripció del producte</p>
  <button>Comprar</button>
</Card>
```

#### Render Props Pattern

Passar funcions que retornen JSX per compartir lògica.

```
interface MouseTrackerProps {
  render: (x: number, y: number) => ReactNode;
}

const MouseTracker = ({ render }: MouseTrackerProps) => {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMove = (e: MouseEvent) => {
      setPosition({ x: e.clientX, y: e.clientY });
    };

    window.addEventListener('mousemove', handleMove);
    return () => window.removeEventListener('mousemove', handleMove);
  }, []);

  return <>{render(position.x, position.y)}</>;
};

// Ús
<MouseTracker render={(x, y) => (
  <p>Posició: {x}, {y}</p>
)} />
```

#### Context API (Estat Global Bàsic)

Per compartir estat entre components sense prop drilling.

```
// 1. Crear context
interface ThemeContextType {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>
(undefined);

// 2. Provider
const ThemeProvider = ({ children }: { children: ReactNode }) => {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };
```

```
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// 3. Custom hook per consumir
const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within ThemeProvider');
  }
  return context;
};

// 4. Ús en components
const Button = () => {
  const { theme, toggleTheme } = useTheme();

  return (
    <button onClick={toggleTheme}>
      Theme actual: {theme}
    </button>
  );
};
```

---

## ✏️ Exercicis Bloc 4

**Exercici 4.1: Comunicació Bàsica Pare-Fill (Bàsic)**

Crea un pare que passi un nom al fill.

**Requisits:**

- Component `Parent` amb un input controlat per `name`
- Component `Child` que rep el nom i el mostra
- Quan escrius al input del pare, s'actualitza al fill en temps real

---

**Exercici 4.2: Callback Fill → Pare (Bàsic)**

El fill notifica el pare quan es fa click.

**Requisits:**

- `Parent` té un comptador d'estat
- `Child` té un botó
- Quan es clica el botó del fill, incrementa el comptador del pare
- El pare mostra el valor del comptador

---

**Exercici 4.3: Lifting State Up - Selectors (Intermedi)**

Dos selects sincronitzats que comparteixen opcions.

**Requisits:**

- Selector 1: Categoria (electronics, clothing, food)
- Selector 2: Subcategoria (depèn de la categoria)
- L'estat viu al pare
- Quan canvia la categoria, es reinicia la subcategoria
- Mostrar selecció actual

---

**Exercici 4.4: Formulari amb Múltiples Inputs (Intermedi)**

Crear un formulari amb components d'input reutilitzables.

**Requisits:**

- Component `Form` (pare) gestiona tot l'estat
- Component `FormInput` (fill) reutilitzable per cada camp
- Props del FormInput: `name`, `value`, `onChange`, `type`, `label`
- Formulari amb: name, email, password
- Validació al submit
- El FormInput és totalment reutilitzable

```
interface FormInputProps {
  name: string;
  value: string;
  onChange: (name: string, value: string) => void;
  type?: string;
  label: string;
}
```

---

**Exercici 4.5: Cistella de Compra (Avançat)**

Sistema complet de productes i cistella.

**Requisits:**

- Component `Shop` (pare) gestiona:
  - Llista de productes
  - Items a la cistella
- Component `ProductList` mostra productes
- Component `ProductCard` per cada producte amb botó "Afegir"
- Component `Cart` mostra items, quantitat i total
- Botó per eliminar de la cistella

- Calcular preu total

**Estructura:**

```
interface Product {
  id: number;
  name: string;
  price: number;
}

interface CartItem {
  product: Product;
  quantity: number;
}
```

---

#### Exercici 4.6: Modal Reutilitzable amb Composició (Intermedi)

Crear un component Modal reutilitzable.

**Requisits:**

- Component `Modal` amb prop `isOpen`, `onClose`, `children`
- Mostrar/ocultar amb estat al pare
- Fons fosc semi-transparent (backdrop)
- Botó X per tancar
- Tancar en clicar el backdrop
- El contingut es passa via `children`
- Exemple d'ús:

```
<Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
  <h2>Títol del Modal</h2>
  <p>Contingut personalitzat aquí</p>
</Modal>
```

---

#### Exercici 4.7: Tabs amb Composició (Avançat)

Sistema de pestanyes reutilitzable.

**Requisits:**

- Component `Tabs` (contenidor)
- Component `Tab` (cada pestanya)
- Component `TabPanel` (contingut de cada pestanya)
- L'estat viu a `Tabs`
- Pestanya activa destacada

- Només es mostra el contingut de la pestanya activa

**Exemple d'ús:**

```
<Tabs>
  <Tab label="Perfil">
    <TabPanel>Contingut del perfil</TabPanel>
  </Tab>
  <Tab label="Configuració">
    <TabPanel>Contingut configuració</TabPanel>
  </Tab>
</Tabs>
```

---

**Exercici 4.8: Context API - Tema Global (Avançat)**

Implementar un sistema de temes amb Context.

**Requisits:**

- Context `ThemeContext` amb theme ('light' | 'dark')
- Provider `ThemeProvider` que envolta l'app
- Custom hook `useTheme`
- Component `ThemeToggle` per canviar tema
- Aplicar classes CSS segons el tema a tota l'app
- Persistir preferència en localStorage

---

**Exercici 4.9: Context + Reducer - Gestor de Tasques (Avançat)**

Sistema complet amb Context i useReducer.

**Requisits:**

- Context `TodoContext`
- useReducer per gestionar accions: ADD, TOGGLE, DELETE, EDIT
- Provider que envolta l'app
- Component `TodoList` (llistar)
- Component `TodoForm` (afegir)
- Component `TodoItem` (individual amb accions)
- Filtres: All, Active, Completed
- Tot desacoblat via context

```
type TodoAction =
  | { type: 'ADD'; payload: string }
  | { type: 'TOGGLE'; payload: number }
  | { type: 'DELETE'; payload: number }
  | { type: 'EDIT'; payload: { id: number; text: string } };
```

```
const todoReducer = (state: Todo[], action: TodoAction): Todo[] => {
  // Implementar
};
```

---

## 📝 Projecte Final Integrador

### BookShelf - Llibreria Personal

Crear una aplicació completa que integri tots els conceptes.

**Funcionalitats:**

1. Llistar llibres (fetch des d'API o mock data)
2. Afegir llibre nou (formulari)
3. Marcar com a llegit/no llegit
4. Filtrar per estat (tots, llegits, per llegir)
5. Cercar per títol o autor (amb debounce)
6. Eliminar llibre
7. Editar llibre (modal)
8. Tema clar/fosc (Context)
9. Persistència amb localStorage

**Estructura de Dades:**

```
interface Book {
  id: number;
  title: string;
  author: string;
  year: number;
  isRead: boolean;
  cover?: string;
}
```

**Components necessaris:**

- `App` (contenidor principal)
- `BookList` (llista)
- `BookCard` (individual)
- `BookForm` (afegir/editar)
- `SearchBar` (cerca amb debounce)
- `FilterButtons` (filtres)
- `Modal` (per editar)
- `ThemeToggle` (canvi de tema)

**Hooks a crear:**

- `useBooks` (gestió de llibres)
- `useDebounce` (cerca retardada)
- `useLocalStorage` (persistència)

**Context:**

- `ThemeContext` per tema global

---

## 🎯 Criteris d'Avaluació

### Nivell Bàsic (5-6)

- Components funcionals correctes
- Props tipades amb TypeScript
- useState bàsic funcional
- Comunicació pare-fill simple

### Nivell Intermedi (7-8)

- Estructura de projecte organitzada
- Gestió d'estat complexa
- useEffect correcte amb dependencies
- Lifting state up
- Composició de components

### Nivell Avançat (9-10)

- Custom hooks reutilitzables
- Context API implementat
- Patrons avançats (render props, compound components)
- Codi net, reutilitzable i ben tipat
- Projecte final complet i funcional

---

## 📚 Recursos Addicionals

**Documentació oficial:**

- React: https://react.dev
- TypeScript: https://www.typescriptlang.org
- Vite: https://vitejs.dev

**Tutorials i guies:**

- React TypeScript Cheatsheet
- Total TypeScript (Matt Pocock)
- Patterns.dev

**APIs per practicar:**

- JSONPlaceholder: https://jsonplaceholder.typicode.com
- Rick and Morty API: https://rickandmortyapi.com
- PokeAPI: https://pokeapi.co

---

**Autor:** Material docent FP DAW - React + TypeScript
**Data:** Febrer 2026
**Versió:** 1.0